



RELATÓRIO TÉCNICO

Atividade 6: Detector de Máximo e Mínimo

ALUNOS

Aluno(a): Tiago Navarro Bettanin

R.A: 2477866

Aluno(a): João Antonio Sitta Martins

R.A: 2553058

1 Introdução

Nesta atividade, o objetivo foi desenvolver, em VHDL, um **detector de valor mínimo e máximo** para uma quantidade genérica de valores **integer**, formados por uma quantidade genérica de bits cada. Diferentemente das atividades anteriores, em que toda a descrição ficava concentrada em uma única entidade, aqui foi necessário organizar o projeto em **duas unidades de compilação**: um *package* em arquivo separado e a entidade principal.

Os principais conceitos novos explorados foram apresentados na Aula 11 e podem ser resumidos da seguinte forma. O **package** é uma unidade de biblioteca destinada a agrupar declarações reutilizáveis — tipos, constantes, funções e procedimentos — de modo que possam ser compartilhados entre projetos. Já o **procedure** é um subprograma com semântica sequencial que, ao contrário de uma *function*, pode **retornar mais de um valor** através de parâmetros do modo *out*; característica fundamental nesta atividade, já que precisamos devolver simultaneamente o mínimo e o máximo do conjunto de entradas.

Adicionalmente, foi exercitada a criação de **tipos definidos pelo usuário**, por meio da declaração de um *array* de **integer** no próprio *package*. O símbolo `<>` aplicado ao *range* permite que o tipo seja irrestrito na declaração e tenha sua dimensão fixada apenas no momento do uso, dando ao tipo a flexibilidade necessária para acomodar qualquer quantidade de entradas. Finalmente, foi reforçado o uso da estrutura **generic**, já trabalhada em atividades anteriores, para parametrizar tanto a quantidade de entradas quanto a largura em bits de cada valor.

2 Desenvolvimento

2.1 Descrição da Lógica Implementada

A solução foi dividida em dois arquivos. O primeiro, *atividade6_pkg.vhd*, contém o *package* com a declaração do tipo *int_array* (um *array* irrestrito de **integer**) e a especificação

e implementação do `procedure find_min_max`. O segundo, `atividade6.vhd`, contém a entidade principal, parametrizada pelos genéricos `num_inputs` (quantidade de valores) e `num_bits` (largura em bits de cada valor).

O algoritmo executado pelo `procedure` é a tradicional **varredura linear**: inicializam-se duas variáveis auxiliares, `temp_min` e `temp_max`, com o primeiro elemento do vetor; em seguida, um laço `for` percorre os demais elementos comparando cada um deles com os valores correntes e atualizando-os quando necessário. Como o vetor é do tipo `int_array` (`integer`), as comparações são feitas diretamente com os operadores `<` e `>`, sem necessidade de conversão para `unsigned`.

Para evitar o erro que o ModelSim apresenta ao declarar um tipo definido pelo usuário como porta de uma entidade, adotou-se a estratégia indicada no enunciado: as portas são declaradas individualmente como `integer` com `range` (`entrada1`, `entrada2`, `entrada3` e `entrada4`) e, dentro da arquitetura, são agrupadas em um sinal `x` do tipo `int_array` por meio de uma atribuição agregada (`aggregate`). Sobre esse sinal interno, o `procedure find_min_max` é chamado de forma concorrente, o que equivale a um processo implícito sensível às entradas e produz lógica combinacional pura após a síntese.

O intervalo dos `integer` foi definido como `0 to 2**num_bits - 1`, exatamente o mesmo padrão utilizado no exemplo `ceil_log2` apresentado na Aula 11. Esse mecanismo garante que a largura em bits do circuito gerado corresponda ao valor de `num_bits`: para `num_bits = 8`, os valores válidos vão de 0 a 255 e o sintetizador utiliza 8 bits para representar cada porta. A entidade fica, portanto, totalmente genérica — basta alterar os dois parâmetros para obter detectores de qualquer dimensão.

2.2 Código VHDL

O Código 1 apresenta o `package` com o tipo `int_array` e o `procedure find_min_max`. Note o uso dos atributos `x'low` e `x'high` no laço `for`: como o vetor é declarado com `range` irrestrito, esses atributos permitem que a varredura funcione corretamente para qualquer dimensão fixada na instanciação, sem precisar fixar limites no próprio `procedure`. As variáveis `temp_min` e `temp_max` são inicializadas com o primeiro elemento (`x(x'low)`) e atualizadas a cada iteração caso o elemento corrente seja menor ou maior do que o valor armazenado. Ao final do laço, os resultados são atribuídos às saídas do `procedure`.

```
1 package atividade6_pkg is
2
3     type int_array is array (natural range <>) of integer;
4
5     procedure find_min_max (
6         signal x          : in  int_array;
7         signal min_val    : out integer;
8         signal max_val    : out integer
9     );
10
11 end package atividade6_pkg;
12
13 package body atividade6_pkg is
14
15     procedure find_min_max (
16         signal x          : in  int_array;
17         signal min_val    : out integer;
18         signal max_val    : out integer
19     ) is
20
21         variable temp_min : integer;
22         variable temp_max : integer;
```

```

23     begin
24
25         temp_min := x(x'low);
26         temp_max := x(x'low);
27
28         for i in x'low + 1 to x'high loop
29             if x(i) < temp_min then
30                 temp_min := x(i);
31             end if;
32             if x(i) > temp_max then
33                 temp_max := x(i);
34             end if;
35         end loop;
36
37         min_val <= temp_min;
38         max_val <= temp_max;
39     end procedure find_min_max;
40
41 end package body atividade6_pkg;

```

Listing 1: Arquivo atividade6_pkg.vhd – package com o tipo definido pelo usuario e o procedure de deteccao.

O Código 2 apresenta a entidade principal. O destaque vai para a linha que agrega as quatro portas individuais no sinal `x` do tipo `int_array`, criando a ponte entre o mundo das portas (simulável pelo ModelSim) e o tipo definido pelo usuário (utilizado internamente pelo *procedure*).

```

1  use work.atividade6_pkg.all;
2
3  entity atividade6 is
4      generic (
5          num_inputs : positive := 4;
6          num_bits   : positive := 8
7      );
8      port (
9          entrada1, entrada2, entrada3, entrada4 :
10             in integer range 0 to 2**num_bits - 1;
11
12             min_out : out integer range 0 to 2**num_bits - 1;
13             max_out : out integer range 0 to 2**num_bits - 1
14      );
15 end entity atividade6;
16
17 architecture atividade6_arch of atividade6 is
18     signal x : int_array(0 to num_inputs - 1);
19
20 begin
21
22     x <= (entrada1, entrada2, entrada3, entrada4);
23
24     find_min_max(x, min_out, max_out);
25
26 end architecture atividade6_arch;

```

Listing 2: Arquivo atividade6.vhd – entidade principal com generic e chamada concorrente do procedure.

2.3 Simulação Funcional

Para validar o circuito, foi realizada uma simulação no ModelSim com a configuração padrão dos genéricos (`num_inputs = 4` e `num_bits = 8`, isto é, 4 entradas com valores no intervalo de 0 a 255). Foram aplicados diferentes conjuntos de valores, incluindo um caso em que todas as entradas são iguais (para verificar que `min_out` e `max_out` produzem o mesmo resultado), um caso em ordem crescente e um caso com os extremos do intervalo. A Tabela ?? lista alguns dos vetores utilizados e os resultados esperados.

Tabela 1: Resultados observados considerando as variações rápidas da entrada4.

entrada1	entrada2	entrada3	entrada4	min_out	max_out
22	14	251	10	10	251
203	86	52	230	52	230
124	153	204	31	31	204
124	153	204	223	124	223
124	153	204	245	124	245
124	153	204	194	124	204
46	167	196	127	46	196
46	167	196	80	46	196
199	219	131	188	131	219
199	219	131	169	131	219
49	38	217	247	38	247
49	38	217	211	38	217
159	143	186	79	79	186
159	143	186	56	56	186

A Figura 1 apresenta a forma de onda obtida no ModelSim. À medida que as entradas variam, as saídas `min_out` e `max_out` acompanham, respectivamente, o menor e o maior valor presente no conjunto das quatro entradas, comprovando o correto funcionamento do detector.

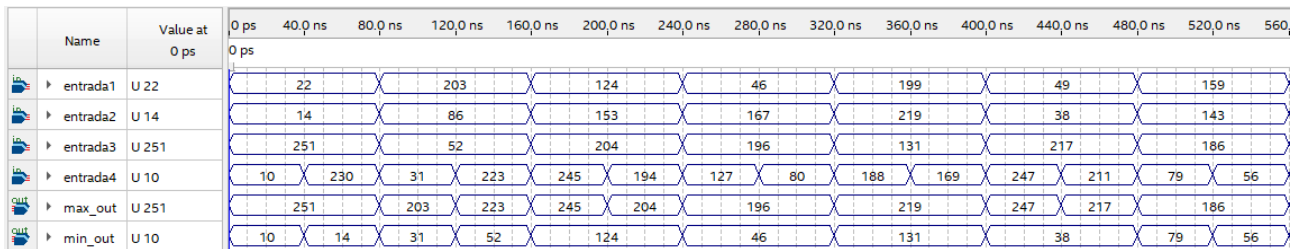


Figura 1: Simulação do detector de máximo e mínimo no ModelSim, para 4 entradas de 8 bits.

2.4 Diagrama RTL

A síntese no Quartus Prime gera o diagrama RTL do circuito, permitindo verificar a estrutura efetivamente produzida a partir do código genérico. Como `num_inputs = 4`, o sintetizador desenrola o laço do `procedure` e gera, para cada uma das saídas, uma cadeia de três comparadores intercalados com multiplexadores que selecionam, a cada estágio, o operando vencedor da comparação. A Figura 2 apresenta o diagrama RTL obtido.

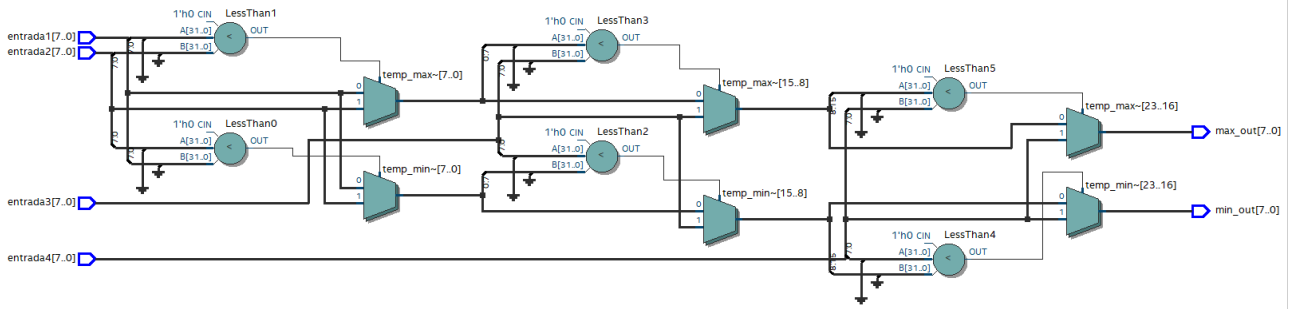


Figura 2: Diagrama RTL do detector de máximo e mínimo, gerado pelo Quartus Prime.

3 Conclusão

A atividade consolidou o uso conjunto dos quatro elementos de biblioteca apresentados na Aula 11: **package**, **procedure**, **generic** e a criação de tipos definidos pelo usuário. A organização do projeto em arquivos separados — *package* contendo a definição reutilizável e a entidade principal contendo a instanciação concreta — evidenciou a vantagem dessa abordagem para projetos de maior porte: o mesmo *package* poderia ser reaproveitado em qualquer outra entidade que precisasse de um detector de mínimo e máximo, bastando declarar um sinal do tipo **int_array** com a dimensão desejada.

A escolha do **procedure** no lugar de uma **function** foi determinante, dado o requisito de retornar dois valores simultaneamente, e o uso de **integer** com *range* permitiu manter o código no nível básico de VHDL praticado em aula, sem necessidade de pacotes adicionais como **numeric_std** nem de conversões entre tipos. A estratégia de declarar portas individuais e agrupá-las em um *aggregate* dentro da arquitetura, sugerida pelo enunciado, mostrou-se eficaz para contornar a limitação do ModelSim em relação a tipos definidos pelo usuário em portas. Com isso, os requisitos propostos para a atividade foram integralmente atendidos [1].

Referências

- [1] Atividade 6: Detector de máximo e mínimo. Documento de enunciado fornecido pelo professor, disciplina de Lógica Reconfigurável, UTFPR Campus Apucarana, 2026/1.